

DQ Compass

Feature Backlog — Delivered Work

MBA-ESG / SG GSC Datathon 2026 — *DQ Compass: Building a Plug-and-Play Data Quality Control Layer for EUCs*

Working Group 1

Generated 2026-09-08

This backlog lists every feature built for the prototype, grouped into 16 epics by technical layer. Each item carries a category tag and a status.

122 items • 121 DONE (built & tested) • 1 PREVIEW (interface ready, implementation partial — JSON / SQL / SharePoint connectors)

Legend

ID	Feature	Category	Status
EX-01	A feature that is built, wired end to end, and covered by a test or a manual check	Backend	DONE
EX-02	A feature whose UI and contracts exist but whose implementation is deferred to phase 2	Backend	PREVIEW

Items per category

Category	Items	What it covers
Backend	64	Python engine, catalogue / source stores, validation, connectors, reporting builders
Full-stack	16	A Flask route together with the server-side view logic behind it
Frontend	21	Jinja templates, the design-token CSS, the vanilla-JS interactions
Platform	10	The dqcompass CLI, packaging (pyproject), the local dev server
Data	1	Seeded demo content (rules, sources)
Docs	5	Architecture / deployment docs and the ReportLab report generators
QA	5	Automated test suites and the dead-code / route checks

Category key: **Backend** Python engine / stores / validation — **Frontend** templates, CSS, JS — **Full-stack** Flask route + its view — **Platform** CLI, packaging, local server — **Data** seed content — **Docs** docs & deployment recipes — **QA** tests & checks.

EPIC A — Control Catalogue

ID	Feature	Category	Status
CAT-01	Canonical catalogue schema: 14 Appendix A.2 attributes + regulatory_ref + technical columns (ref_dataset_scope, params JSON, threshold_pct, active, timestamps)	Backend	DONE
CAT-02	Atomic CSV storage (CatalogueStore, temp file + os.replace)	Backend	DONE
CAT-03	Rule CRUD: create / update / delete / activate-deactivate (upsert, delete, set_active)	Backend	DONE
CAT-04	Append-only audit log catalogue_changelog.jsonl: SHA-256 of the file before/after, rule before/after, user, note	Backend	DONE
CAT-05	Per-rule reconstructable history (history(rule_id))	Backend	DONE
CAT-06	Source-rename cascade: retarget_dataset(old, new) rewrites dataset_scope / ref_dataset_scope on every dependent rule	Backend	DONE
CAT-07	Auto-generated auditor-readable columns: logic_definition (pseudo-SQL), data_element, threshold (text) on every save	Backend	DONE
CAT-08	Plain-language explanation of each rule (explain_plain_language) for the form preview	Backend	DONE
CAT-09	Automatic ID assignment (next_rule_id -> DQ08, DQ09...)	Backend	DONE
CAT-10	7 seeded demo rules DQ01-DQ07 (DQ07 carries a BCBS 239 regulatory_ref)	Data	DONE

EPIC B — Meta-catalogue / validation engine

ID	Feature	Category	Status
VAL-01	Controlled vocabularies: control types, severities, frequencies, output types	Backend	DONE
VAL-02	validate_rule(): ~30 blocking (errors) + non-blocking (warnings/infos) checks before a rule enters the catalogue	Backend	DONE
VAL-03	Per-control-type parameter validation (PARAM_SPEC, validate_params): columns exist, single validity criterion, no duplicate keys, regex compiles, numeric bounds coherent, tolerance 0-100	Backend	DONE
VAL-04	Cross-field checks: High severity + low frequency, loose threshold on a critical control, duplicate logic between two rules	Backend	DONE
VAL-05	Structured ValidationReport (errors / warnings / infos / ok) consumed by the UI and the CLI	Backend	DONE
VAL-06	Help dictionaries (FIELD_HELP, CONTROL_TYPE_HELP, OUTPUT_TYPE_HELP, SEVERITY_HELP, FREQUENCY_HELP) - single source for tooltips AND the /docs page	Backend	DONE

EPIC C — Connectors / data sources

ID	Feature	Category	Status
CON-01	Connector abstraction: load() / probe() / validate() contract per type; SourceUnavailable, Field, ProbeResult, SourceIssue	Backend	DONE
CON-02	CsvConnector: path, delimiter (incl. tab), header row, encoding, quotechar, skip rows	Backend	DONE
CON-03	ExcelConnector: sheet, header row, skip_rows, nrows (closes file handle cleanly)	Backend	DONE
CON-04	JsonConnector / SqlConnector / UrlConnector (SharePoint/Drive/URL): _PlannedConnector - selector + shape validation, load() raises SourceUnavailable	Backend	PREVIEW
CON-05	Registry + API: CONNECTOR_ORDER, get_connector(), describe_connectors()	Backend	DONE
CON-06	normalize_entry(): accepts the legacy flat {path, format} and the rich shape	Backend	DONE
CON-07	load_source(name, config, root): raises SourceUnavailable if source missing or not active	Backend	DONE

ID	Feature	Category	Status
CON-08	Rich datasets_config.yaml format: {type, status (active/retired/preview), config, timestamps}	Backend	DONE
CON-09	Test-connection (probe) from the UI with shape feedback (detected columns, rows)	Full-stack	DONE

EPIC D — Source deletion policy

ID	Feature	Category	Status
SRC-01	used_by(name): list of rules depending on a source	Backend	DONE
SRC-02	Retire (reversible soft delete): status -> retired + cascade-deactivate rules + log	Backend	DONE
SRC-03	Reactivate: re-enables the source, returns still-inactive rules; reactivate_rules() to re-enable them selectively	Backend	DONE
SRC-04	Purge (permanent delete, typed confirmation)	Backend	DONE
SRC-05	Rename: calls retarget_dataset on the catalogue (CAT-06 cascade)	Backend	DONE
SRC-06	Engine resilience: a rule on a missing/retired source -> status: ERROR record in the evidence, run continues	Backend	DONE
SRC-07	Hash-chained data_sources_changelog.jsonl	Backend	DONE
SRC-08	validate_source() + plain_summary() (source meta-catalogue)	Backend	DONE

EPIC E — DQ Engine (core)

ID	Feature	Category	Status
ENG-01	6 generic control functions (completeness/validity/uniqueness/consistency/timeliness/reconciliation), fixed contract -> {status, metrics, exceptions}	Backend	DONE
ENG-02	CONTROL_REGISTRY: control_type -> function mapping	Backend	DONE
ENG-03	Honours active=FALSE (rule skipped) and threshold_pct	Backend	DONE
ENG-04	_apply_threshold(): FAIL -> PASS when exception rate <= threshold; exposes exception_pct, threshold_pct, within_threshold; shared by execution + verification	Backend	DONE
ENG-05	_resolve_execution_parameters(): resolves reference_date: "today" -> real ISO date, etc.	Backend	DONE
ENG-06	Severity-weighted composite DQ score (High=5 / Medium=2 / Low=1; per-rule penalty; score = 100*(1 - sum penalty / sum weight), clamped [0,100])	Backend	DONE
ENG-07	Deterministic, reproducible execution (dedicated test)	Backend	DONE
ENG-08	_error_record(): unknown control type or unavailable source -> ERROR, run continues	Backend	DONE
ENG-09	CLI python dq_engine.py --verify (non-zero exit on divergence)	Platform	DONE

EPIC F — Audit Layer / Evidence Pack

ID	Feature	Category	Status
AUD-01	Evidence Pack per run - the 10 Appendix B.2 components	Backend	DONE
AUD-02	run_summary.json with previous_run_hash / run_hash = SHA-256(prev + canonical_json(summary))	Backend	DONE
AUD-03	Append-only hash-chained ledger evidence/runs_ledger.jsonl	Backend	DONE
AUD-04	Exact snapshot of each input dataset (snapshots/.csv + hash)	Backend	DONE
AUD-05	Per rule: _result.json (config snapshot + resolved execution parameters) + _exceptions.csv	Backend	DONE

ID	Feature	Category	Status
AUD-06	system_trace.json: engine/python/platform/host versions, pandas/pyyaml/openpyxl versions, catalogue/config/controls.py hashes, control function per rule, retention note (WORM in prod)	Backend	DONE
AUD-07	Logs: run_log.jsonl (structured) + run.log (text) via RunLog	Backend	DONE
AUD-08	verify_run(evidence_dir, run_id): re-executes stored config against stored snapshots, compares status + exception_count, writes verification.json (all_match)	Backend	DONE
AUD-09	Sign-off: signoff.json written from the Runs page	Full-stack	DONE

EPIC G — Reporting Layer

ID	Feature	Category	Status
REP-01	scorecard.csv + scorecard.html (traffic lights, GREY for ERROR)	Backend	DONE
REP-02	exceptions_detail.csv (resolves the relative evidence path)	Backend	DONE
REP-03	coverage.csv - datasets x dimensions coverage	Backend	DONE
REP-04	exception_summary.csv - long format, dimension in {severity, owner, control_type, dataset}	Backend	DONE
REP-05	dataset_score.csv - severity-weighted score per dataset	Backend	DONE
REP-06	trend.csv - history from the ledger	Backend	DONE
REP-07	alerts.json - failing High-severity rules + remediation_action from the catalogue	Backend	DONE
REP-08	overall_dq_score(run_dir) + run_result_map(run_dir)	Backend	DONE
REP-09	Robust _latest_run_dir() (skips the ledger file)	Backend	DONE

EPIC H — Supervisory Mapping / governance

ID	Feature	Category	Status
MAP-01	build_mapping() - Appendix C: one row per active rule (requirement, control_implemented, evidence_generated, output, kpi, last_run_id/status/hash)	Backend	DONE
MAP-02	regulatory_ref column (BCBS 239) in the schema; default derived per DQ dimension when blank	Backend	DONE
MAP-03	supervisory_mapping.csv export (Appendix C.2 / C.3)	Backend	DONE
MAP-04	PRINCIPLES: Traceability, Repeatability, Auditability, Governance	Backend	DONE

EPIC I — Web application / Flask routes

ID	Feature	Category	Status
APP-01	/ - Home dashboard (health tiles, latest-run donut, attention list, coverage heatmap, recent activity)	Full-stack	DONE
APP-02	/catalogue - filterable, expandable rule list	Full-stack	DONE
APP-03	/rule/new, /rule/ - create/edit form with live validation	Full-stack	DONE
APP-04	/sources, /sources/* - CRUD + retire / reactivate / reactivate-rules / purge / rename / probe	Full-stack	DONE
APP-05	/runs, /runs/ - navigable Evidence Pack, /runs/verify, /signoff, /file/	Full-stack	DONE
APP-06	/run-now - triggers an engine execution	Full-stack	DONE
APP-07	/reporting - Reporting & Analytics page (see EPIC K)	Full-stack	DONE
APP-08	/mapping - supervisory mapping	Full-stack	DONE
APP-09	/activity - merged catalogue + sources audit timeline	Full-stack	DONE

ID	Feature	Category	Status
APP-10	/docs - built-in documentation (control types, output types, severities...)	Full-stack	DONE
APP-11	/download/* - reporting CSV / JSON exports	Full-stack	DONE
APP-12	Helpers: sources_index, dataset_columns, rule_reports, latest_run, merged_activity, list_runs, load_run, reporting_context	Backend	DONE
APP-13	"Editor" field (user name) in the topbar, injected into the audit logs	Full-stack	DONE
APP-14	Local server run_authoring.py on 127.0.0.1:5001	Platform	DONE

EPIC J — Frontend: shell, design system, pages

ID	Feature	Category	Status
FE-01	App shell: sidebar (Home / Catalogue / Data Sources / Runs / Reporting / Mapping / Activity / Help) + topbar	Frontend	DONE
FE-02	Design-token system style.css: :root light, [data-theme="dark"], @media prefers-color-scheme	Frontend	DONE
FE-03	Light/dark theme toggle persisted (localStorage)	Frontend	DONE
FE-04	Inter font, inline-SVG iconography via _ui.html macro (icon, page_header, stat, sev_pill, status_dot, conn_badge)	Frontend	DONE
FE-05	catalogue.html - expandable list rows, chip filters with live counts (.fc-count, .is-zero)	Frontend	DONE
FE-06	form.html - required fields marked, hover ? help icons, dynamic parameter fields per control type	Frontend	DONE
FE-07	sources.html / source_form.html - source cards + tile connector picker + Test-connection button	Frontend	DONE
FE-08	runs.html / run_detail.html - Evidence Pack (.kv, .segbar), verify, sign-off	Frontend	DONE
FE-09	dashboard.html, mapping.html, activity.html (timeline), docs.html	Frontend	DONE
FE-10	base.html: app shell, _ui.html macro import, per-page breadcrumb	Frontend	DONE
FE-11	Critical fix [hidden]{display:none!important} (so el.hidden wins over .row-item{display:grid})	Frontend	DONE
FE-12	Responsive: mobile sidebar, scroll-x containers for wide tables / heatmaps	Frontend	DONE
FE-13	Full translation of the application to English	Frontend	DONE

EPIC K — Frontend: Reporting & Analytics

ID	Feature	Category	Status
RPT-01	Audience filter: All / Control owners & risk / Data engineers (data-aud / data-aud-set, JS sets data-aud-view)	Frontend	DONE
RPT-02	charts.py - dependency-free charts (inline SVG/HTML, token-coloured): score_bar, trend_chart, donut, hbars, sparkline, mini_bar	Full-stack	DONE
RPT-03	Business half: composite DQ score + delta + sparkline, score bar (70/90 bands), trend chart (score line + pass/fail/error strips, auto-floored Y axis), dataset-health ranking, 3-state coverage matrix, open High-severity issues + remediation, exceptions by owner	Frontend	DONE
RPT-04	Data-engineer half: exceptions by dimension, top offending rules, per-rule reliability table (fail-rate mini-bars), "what changed since previous run" diff, ERROR / broken-controls list	Frontend	DONE
RPT-05	CSV / JSON downloads from the page	Full-stack	DONE
RPT-06	reporting_context(): assembles score/delta, pass_rate, KPIs, trend[12], reliability[12], by_dim/by_owner/by_rule, 3-state coverage, run age	Backend	DONE
RPT-07	Cross-links from Home and run_detail to /reporting	Frontend	DONE

ID	Feature	Category	Status
RPT-08	Reporting CSS classes: .sec-head, .kpi-delta, .score-*, .chart-svg, .hb-*, .cov3 (hardened with color-mix), .alert-item, .chg	Frontend	DONE

EPIC L — Frontend: JS interactions

ID	Feature	Category	Status
JS-01	app.js: theme toggle, mobile sidebar, list filters with per-chip counts, [data-expand] row expand, audience filter	Frontend	DONE
JS-02	form.js: dynamic parameter fields per control type, rule preview	Frontend	DONE
JS-03	source_form.js: tile connector picker + Test-connection probe	Frontend	DONE

EPIC M — CLI & packaging

ID	Feature	Category	Status
CLI-01	pyproject.toml + pip install -e . (dqcompass console script)	Platform	DONE
CLI-02	dqcompass run - executes the engine	Platform	DONE
CLI-03	dqcompass verify - independent verification	Platform	DONE
CLI-04	dqcompass report - regenerates the reporting outputs	Platform	DONE
CLI-05	dqcompass mapping - regenerates the supervisory mapping	Platform	DONE
CLI-06	dqcompass catalogue-validate - validates the whole catalogue	Platform	DONE
CLI-07	dqcompass gate --severity High - non-zero exit code to block a CI pipeline	Platform	DONE
CLI-08	dqcompass serve - starts the web app	Platform	DONE

EPIC N — Deployment & documentation

ID	Feature	Category	Status
DOC-01	docs/ARCHITECTURE.md - architecture, separation of concerns	Docs	DONE
DOC-02	docs/DEPLOYMENT.md - cron / Airflow / CI gate / notification recipes	Docs	DONE
DOC-03	docs/build_report.py -> DQ_Compass_Implementation_and_Compliance.pdf (ReportLab, 24 pages: per-component detail + line-by-line brief-compliance matrix, Appendices A.2 / B.2-B.3 / C)	Docs	DONE
DOC-04	README.md - 8-section table	Docs	DONE
DOC-05	requirements.txt (pandas, pyyaml, flask, openpyxl) + .gitignore	Docs	DONE

EPIC O — Data migration

ID	Feature	Category	Status
MIG-01	migrate_catalogue.py - old CSV -> 20-column canonical schema	Backend	DONE
MIG-02	migrate_sources.py - flat datasets_config.yaml -> rich format (engine reads both)	Backend	DONE

EPIC P — Tests / QA

ID	Feature	Category	Status
QA-01	tests/test_schema.py - 21 tests (validation, derivations, vocabularies)	QA	DONE

ID	Feature	Category	Status
QA-02	tests/test_connectors.py - 11 tests (normalize_entry, CSV/Excel, planned connectors)	QA	DONE
QA-03	tests/test_engine.py - 6 tests (Evidence Pack structure, reproducibility + ledger chain, verify_run, resolved parameters, threshold FAIL->PASS, missing source -> ERROR)	QA	DONE
QA-04	Dead-code sweep with pyflakes (exit 0) - unused imports / functions removed	QA	DONE
QA-05	Manual check: 8 routes return 200, audience toggle + charts OK in light and dark	QA	DONE

Notes for the jury

- The four brief lifecycle stages — Definition, Execution, Reporting, Audit — are each covered by a dedicated epic (A/B, E, G, F) and are wired end to end.
- The only deferred work is three of five connector types (JSON, SQL, SharePoint/Drive/URL). Their selector, validation and storage exist; only load() is stubbed. CSV and Excel run.
- Everything under EPIC F (Audit) and EPIC H (Supervisory Mapping) is what turns a set of checks into auditable governance: hash-chained ledgers, independent re-execution, and an auto-generated Appendix C mapping.
- 38 automated tests (21 + 11 + 6) plus a pyflakes-clean codebase and an 8-route manual pass.